

Of course, one can argue that this is where the database admin walks in and flexes his muscles looking all-important and goes about doing his routine backups and optimization. Alternatively, you can leverage one of the many services on offer from AWS that does many of these for you. Let us take a look at what's on offer and the advantages they provide.

Relational Database Service (RDS)

AWS offers a completely managed version of multiple databases as a service. It is completely resizable and varies on the data you store such that you do not need to provision machines for it separately. You also end up paying only for the storage space and I/O operations that you use and not for the underlying VMs that are used. In addition to that, most database administration tasks are handled by Amazon itself including regular backups, point in time recoveries and trigger optimizations.

This allows the user to focus more on the data and its usage rather than getting tangled up in non-functional maintenance activities. Let us take a look at how to deploy and connect to a database instance on RDS.

Setup a database on RDS

- Open the “Services” tab from the AWS Console and navigate to ‘RDS’ under Databases.
- This should open up the Amazon RDS Console and default to the Dashboard. The dashboard lists all the running instances of RDS, click on any of them to view their details.
If there are none running, an additional popup introducing RDS opens up with the button “Get Started Now”. Click on this to open up a wizard that creates and launches your first RDS instance.
- The first screen on the wizard allows you to select the underlying database engine. AWS currently supports six database engines: Aurora, Oracle, PostgreSQL, MySQL, MariaDB and Microsoft SQL Server. Select any one of these according to your requirement and click ‘Next’. [Please note: Aurora is not supported under the ‘free tier’ and will be billed as per usage]
- The next screen brings up the DB details like engine version, licenses to be used, as well as the size of the underlying db instance (micro, medium, large etc). You can choose to select a multi-availability zone deployment as well which would allow your database to become more fault tolerant with lower latency. Next, select the storage class you want to use, (SSD